

Git Repository Structure and Management

Ryc Brownrigg

March 2026

Contents

1	Overview	2
2	How the Submodule Was Set Up	2
3	Day-to-Day Workflow	2
3.1	Working in the Parachain Repo Only (Most Common)	2
3.2	Working in the Root Repo Only	3
3.3	Updating the Submodule Pointer in the Root Repo	3
3.4	Full Workflow (Both Repos Changed)	3
4	Cloning the Project (For Examiners or Collaborators)	4
5	Remote Repository Management	4
5.1	Two Independent GitHub Repositories	4
5.2	Pushing Changes	4
5.3	Branch Management	4
6	Future Merge Into a Single Repository	5
6.1	Option A: Absorb the Submodule (Recommended)	5
6.2	Option B: Move Everything Into the Parachain Repo	5
6.3	Recommendation	6
7	Quick Reference	6

1 Overview

This project uses two git repositories managed via a **git submodule** relationship:

Repository	Purpose	GitHub Remote
Root	Thesis documentation, phase deliverables, project-level config	RycBrownrigg/Cross-Chain_Content_Rights_Management_Service.git
Submodule	Parachain source code (runtime, pallets, node, contracts)	RycBrownrigg/content-rights-parachain.git

The root repo tracks the parachain repo as a git submodule, meaning it stores a **pointer to a specific commit** in the parachain repo rather than a copy of the files.

2 How the Submodule Was Set Up

The `content-rights-parachain` directory was originally cloned independently inside the root project directory. It had its own `.git/` directory, commit history, and GitHub remote. The root repo saw it as an untracked directory.

To formalise the relationship, the following command was run from the root repo:

```
git submodule add
↳ https://github.com/RycBrownrigg/content-rights-parachain.git
↳ content-rights-parachain
```

This created two things in the root repo:

1. `.gitmodules` — a configuration file recording the submodule's path and remote URL:

```
[submodule "content-rights-parachain"]
  path = content-rights-parachain
  url = https://github.com/RycBrownrigg/content-rights-parachain.git
```

2. **A submodule reference** — the root repo's index now tracks which commit hash in the parachain repo corresponds to the current thesis state.

Both were committed to the root repo with:

```
git add .gitmodules content-rights-parachain
git commit -m "Add content-rights-parachain as git submodule"
```

3 Day-to-Day Workflow

3.1 Working in the Parachain Repo Only (Most Common)

When making code changes to the parachain:

```
cd content-rights-parachain
# ... make changes ...
git add <files>
git commit -m "Description of changes"
git push origin main
```

The root repo will now show `content-rights-parachain` (modified content) or (new commits) in `git status`. This is expected — it means the submodule has moved ahead of the pointer the root repo is tracking.

3.2 Working in the Root Repo Only

When updating thesis documentation:

```
# From root directory
git add <files>
git commit -m "Description of changes"
git push origin main
```

This does not affect the parachain repo at all.

3.3 Updating the Submodule Pointer in the Root Repo

After committing in the parachain repo, update the root repo's reference so it points to the latest parachain commit:

```
# From root directory
git add content-rights-parachain
git commit -m "Update content-rights-parachain submodule reference"
git push origin main
```

This records in the root repo's history exactly which parachain commit corresponds to the current state of the thesis documentation.

3.4 Full Workflow (Both Repos Changed)

```
# 1. Commit parachain changes first
cd content-rights-parachain
git add <files>
git commit -m "Implement content-rights pallet"
git push origin main

# 2. Return to root, commit thesis changes and update submodule pointer
cd ..
git add "Phase 4 - Core Development & Implementation (Weeks 13-16)/..."
git add content-rights-parachain
git commit -m "Add Phase 4 deliverables and update parachain submodule"
git push origin main
```

4 Cloning the Project (For Examiners or Collaborators)

Anyone cloning the root repo needs to initialise the submodule to get the parachain code:

```
# Clone with submodule in one step
git clone --recurse-submodules \

    ↪ https://github.com/RycBrownrigg/Cross-Chain_Content_Rights_Management_Service.git

# Or clone first, then initialise
git clone
    ↪ https://github.com/RycBrownrigg/Cross-Chain_Content_Rights_Management_Service.git
cd Cross-Chain_Content_Rights_Management_Service
git submodule update --init
```

Without `--recurse-submodules` or `git submodule update --init`, the `content-rights-parachain` directory will exist but be empty.

5 Remote Repository Management

5.1 Two Independent GitHub Repositories

Each repo has its own GitHub remote and can be managed independently:

- **Root repo:** https://github.com/RycBrownrigg/Cross-Chain_Content_Rights_Management_Service.git
- **Parachain repo:** <https://github.com/RycBrownrigg/content-rights-parachain.git>

5.2 Pushing Changes

Each repo is pushed separately:

```
# Push parachain changes
cd content-rights-parachain
git push origin main

# Push root changes (including submodule pointer updates)
cd ..
git push origin main
```

5.3 Branch Management

Both repos use `main` as the primary branch. If feature branches are needed, create them in the relevant repo:

```
# Feature branch in parachain repo
cd content-rights-parachain
git checkout -b feature/content-rights-pallet
```

```
# ... work, commit ...
git push origin feature/content-rights-pallet
# Merge via PR on GitHub, then update root submodule pointer
```

6 Future Merge Into a Single Repository

At some point (likely after thesis submission), it may be desirable to merge both repos into a single repository. There are two approaches.

6.1 Option A: Absorb the Submodule (Recommended)

This removes the submodule relationship and brings the parachain code directly into the root repo, preserving all commit history from both repos.

```
# From root directory

# 1. Remove the submodule registration
git submodule deinit content-rights-parachain
git rm content-rights-parachain
rm -rf .git/modules/content-rights-parachain

# 2. Merge the parachain repo's history using subtree
git subtree add --prefix=content-rights-parachain \
    https://github.com/RycBrownrigg/content-rights-parachain.git main

# 3. Commit and push
git push origin main
```

After this:

- All parachain files live in `content-rights-parachain/` as regular tracked files
- The parachain repo's full commit history is preserved in the root repo
- Only one remote repo needs to be maintained going forward
- The separate parachain GitHub repo can be archived

6.2 Option B: Move Everything Into the Parachain Repo

If the code repo is considered the primary artifact:

```
# From parachain repo
# Copy thesis docs into a docs/ or thesis/ subdirectory
# Commit and push
# Archive the root repo
```

This is simpler but loses the root repo's commit history unless a subtree merge is used in reverse.

6.3 Recommendation

Option A is preferred because:

- The root repo already has the thesis documentation history
- The submodule relationship means the root repo is the natural “parent”
- `git subtree add` preserves the full parachain commit history
- The result is a single repo with complete history from both sources

Timing: Do this after thesis submission to avoid disrupting the workflow during active development. The submodule setup works well for the development phase where code and documentation change at different rates.

7 Quick Reference

Task	Command(s)
Check root repo status	<code>git status</code> (from root)
Check parachain status	<code>cd content-rights-parachain && git status</code>
Commit parachain changes	<code>cd content-rights-parachain && git add <files> && git commit -m "msg" && git push</code>
Update submodule pointer	<code>git add content-rights-parachain && git commit -m "Update submodule ref"</code> (from root)
Clone with submodule	<code>git clone --recurse-submodules <root-repo-url></code>
Initialise submodule after clone	<code>git submodule update --init</code>
See which parachain commit root tracks	<code>git submodule status</code>
Pull latest parachain changes	<code>cd content-rights-parachain && git pull origin main</code>